

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 4

Comparative Study of Deep Convolutional Neural Network Architectures Using Transfer Learning

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

Code Repository: https://github.com/tonypanda7/DeepLearning_Lab

1. Objective

The objectives of this experiment are

- Study the evolution of deep CNN architectures.
- Compare LeNet-5, AlexNet, VGG16, GoogleNet and ResNet.
- Understand transfer learning.
- Fine tune pretrained CNN models.
- Compare classification performance of different architectures.

2. Learning Outcomes

After completing this experiment students will be able to

1. Explain modern CNN architectures.
2. Differentiate LeNet, AlexNet, VGG16, GoogleNet and ResNet.
3. Implement transfer learning.
4. Fine tune pretrained CNN models.
5. Compare CNN architectures using performance metrics.

3. Introduction

Deep Convolutional Neural Networks have become the dominant models for computer vision applications because they automatically learn hierarchical features from images. The evolution of CNNs has focused on improving classification accuracy while reducing computational complexity and training difficulties.

The progression of CNN architectures began with LeNet-5 and has evolved through AlexNet, VGGNet, GoogleNet, and ResNet. Each architecture introduced important innovations that significantly improved image classification performance.

4. Evolution of CNN Architectures

Model	Year	Major Contribution
LeNet-5	1998	First practical convolutional neural network
AlexNet	2012	ReLU activation, Dropout and GPU training
VGG16	2014	Uniform 3×3 convolution filters
GoogleNet	2014	Inception modules for multi-scale feature extraction
ResNet	2015	Residual learning using skip connections

5. LeNet-5

LeNet-5 was proposed by Yann LeCun in 1998 for handwritten digit recognition. It consists of two convolution layers, two average pooling layers and fully connected layers. It demonstrated that convolutional networks could automatically learn image features without handcrafted feature extraction.

Advantages

- Small number of parameters
- Fast training
- Suitable for OCR

6. AlexNet

AlexNet won the ImageNet Challenge in 2012 by reducing the classification error significantly. It introduced ReLU activation, Dropout regularization and GPU training.

Major innovations

- ReLU activation
- Dropout
- GPU implementation
- Data augmentation

7. VGG16

VGG16 demonstrated that using multiple small 3×3 convolution filters could outperform larger filters while increasing network depth.

Advantages

- Excellent feature extraction
- Simple architecture
- High classification accuracy

8. Comparison of Architectures

Architecture	Depth	Parameters	Main Contribution
LeNet-5	7	60K	First CNN
AlexNet	8	61M	ReLU + Dropout
VGG16	16	138M	Deep 3×3 filters
GoogleNet	22	6.8M	Inception Modules
ResNet50	50	25.6M	Residual Learning

9. GoogleNet (Inception Network)

GoogleNet, proposed by Szegedy *et al.* in 2014, introduced the **Inception Module**, which performs multiple convolution operations in parallel using different filter sizes. Instead of using only a single kernel size, GoogleNet simultaneously applies 1×1 , 3×3 , 5×5 convolutions and max pooling. The outputs are concatenated to obtain multi-scale feature representations.

The use of 1×1 convolutions before larger kernels reduces the number of trainable parameters and computational complexity.

Advantages

- Multi-scale feature extraction.
- Fewer parameters than VGG16.
- High computational efficiency.
- Better classification performance.

10. ResNet

Increasing CNN depth often causes the **vanishing gradient problem**, making optimization difficult. ResNet solves this issue using **skip (residual) connections**, allowing gradients to flow directly through the network.

Instead of learning

$$H(x)$$

ResNet learns

$$F(x) = H(x) - x$$

thus,

$$Output = F(x) + x$$

where

- $F(x)$ = Residual Mapping
- x = Identity Mapping

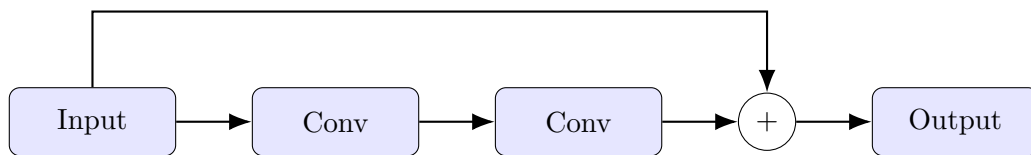


Figure 1: Residual Block in ResNet

Advantages

- Eliminates vanishing gradients.
- Enables very deep neural networks.
- Faster convergence.
- High ImageNet accuracy.

11. Dilated Convolution

Dilated convolution (also called atrous convolution) increases the receptive field without increasing the number of parameters.

The dilation rate determines the spacing between kernel elements.

For a standard convolution,

$$D = 1$$

For dilated convolution,

$$D > 1$$

Example

$$\begin{bmatrix} 1 & 0 & 2 \\ 0 & 3 & 0 \\ 4 & 0 & 5 \end{bmatrix}$$

where zeros indicate inserted gaps.

Applications

- Semantic segmentation
- Medical image analysis
- Satellite imagery
- Object localization

12. Transpose Convolution

Transpose convolution increases the spatial dimensions of feature maps. Unlike pooling, which reduces image size, transpose convolution performs learnable upsampling.

Applications

- Image Super Resolution
- Autoencoders
- GANs
- Image Generation
- Semantic Segmentation

13. Transfer Learning

Transfer Learning utilizes a pretrained CNN model that has already learned general image features from a large dataset such as ImageNet.

Instead of training from scratch,

1. Load a pretrained model.
2. Remove the original classifier.
3. Freeze convolution layers.
4. Add new Dense layers.
5. Train the classifier.
6. Fine tune selected convolution layers.

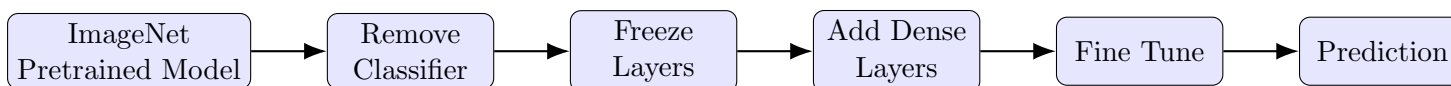


Figure 2: Transfer Learning Workflow

Advantages

- Faster convergence.
- Higher accuracy on small datasets.
- Reduced training time.
- Lower computational cost.

14. Dataset

Dataset: CIFAR-10

- Training Images : 50,000
- Testing Images : 10,000
- Number of Classes : 10
- Image Size : $32 \times 32 \times 3$
- Classes: Airplane, Automobile, Bird, Cat, Deer, Dog, Frog, Horse, Ship and Truck.

15. Experimental Procedure

Task 1: Dataset Preparation

1. Load the CIFAR-10 dataset using TensorFlow/Keras.
2. Normalize the pixel values to the range $[0,1]$.
3. Display ten sample images.
4. Print the dimensions of the training and testing datasets.

Task 2: Transfer Learning

Load any one of the following pretrained CNN models.

- VGG16
- ResNet50
- MobileNetV2
- EfficientNetB0 (Optional)

Perform the following steps.

1. Load pretrained ImageNet weights.
2. Remove the original classification layer.
3. Freeze the convolutional base.
4. Add a Global Average Pooling layer.
5. Add a Dense layer with ReLU activation.
6. Add the output layer with Softmax activation.

Task 3: Model Training

Train the model using

Optimizer	Adam
Learning Rate	0.001
Batch Size	32
Epochs	10–20
Loss Function	Categorical Cross Entropy
Evaluation Metric	Accuracy

Task 4: Fine Tuning

1. Unfreeze the last convolution block.
2. Train for another 5–10 epochs.
3. Compare the classification accuracy before and after fine tuning.

Task 5: Model Evaluation

Evaluate the trained model using

- Accuracy

- Precision
- Recall
- F1-score
- Confusion Matrix
- Classification Report

16. Hyperparameter Study

Compare the performance using different settings.

Hyperparameter	Values
Learning Rate	0.001, 0.0001
Batch Size	16, 32, 64
Epochs	10, 20
Optimizer	Adam, SGD
Dense Units	128, 256
Frozen Layers	All, Partial

17. Mandatory Plots

The following plots were obtained while implementing the VGG16 based transfer learning model on the CIFAR-10 dataset.



Figure 3: Sample CIFAR-10 Images

Inference: The dataset comprises $32 \times 32 \times 3$ colour images divided into ten balanced classes. Due to the low resolution of the images, fine-grained classification is challenging, which motivates the use of pretrained feature extractors.

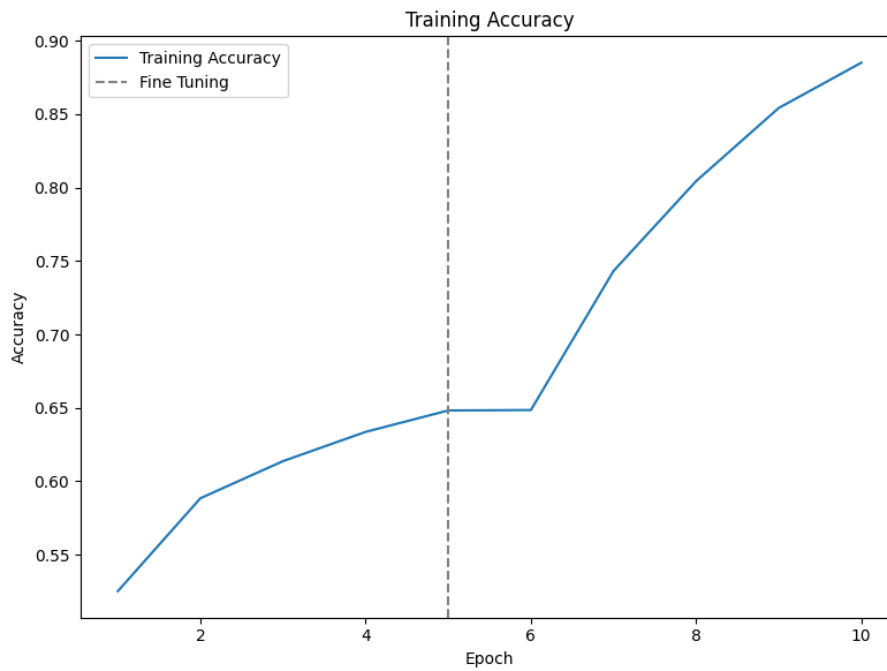


Figure 4: Training Accuracy (Initial Training + Fine Tuning)

Inference: The plot shows the increase in accuracy during initial training with the frozen base: it goes up steadily from 52.5% to 64.8%. Once we unfreeze the `block5` layers (dashed line), the accuracy spikes up to 88.5%—signifying that fine tuning allowed the network to learn additional dataset-specific features.

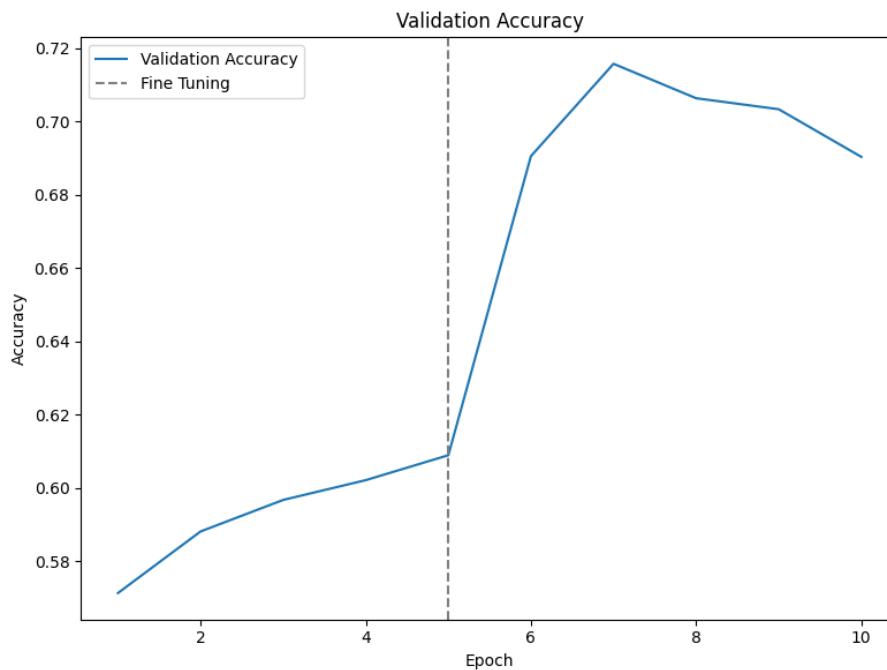


Figure 5: Validation Accuracy (Initial Training + Fine Tuning)

Inference: The validation accuracy increases from 57.16% to 71.6% after fine tuning. The gap that opens up between training and validation accuracy in later fine-tuning epochs indicates the onset of overfitting.

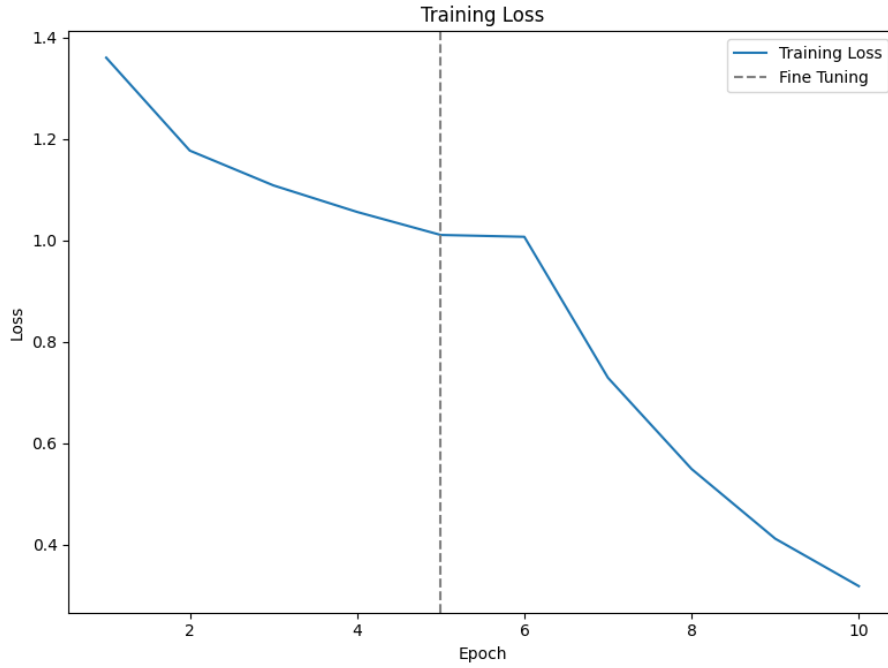


Figure 6: Training Loss (Initial Training + Fine Tuning)

Inference: Training loss goes down steadily over time, falling off rapidly after the start of the fine-tuning process (from around 1.01 to 0.32), demonstrating that the model is beginning to fit the training set more accurately once deeper layers have been made trainable.

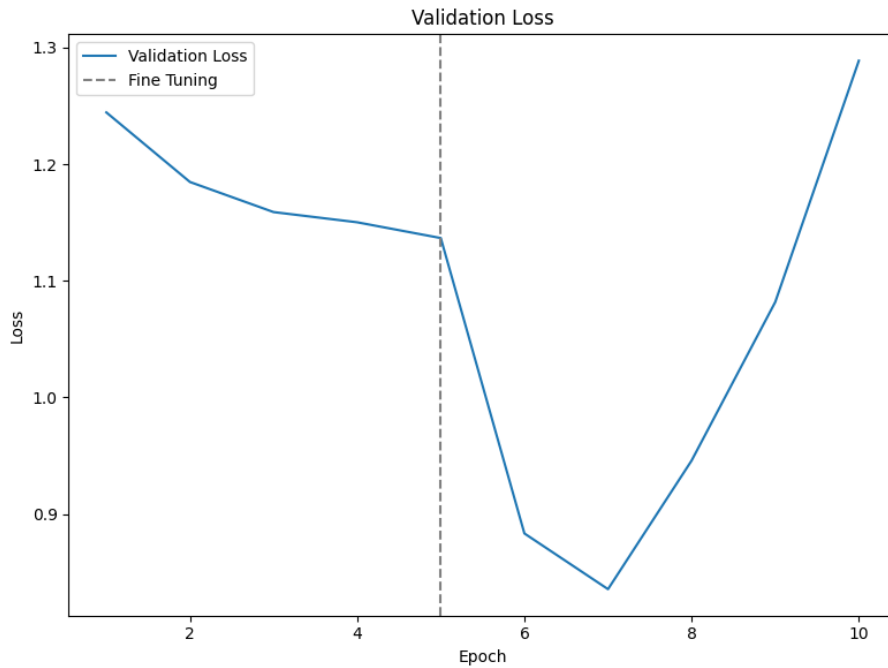


Figure 7: Validation Loss (Initial Training + Fine Tuning)

Inference: The validation loss decreases during the initial training and first fine-tuning epoch, dropping to a minimum of about 0.84, but then increases in later fine-tuning epochs. This diverging pattern (increasing loss while training loss continues to decrease) is a telltale sign of overfitting now that more layers are unfrozen.

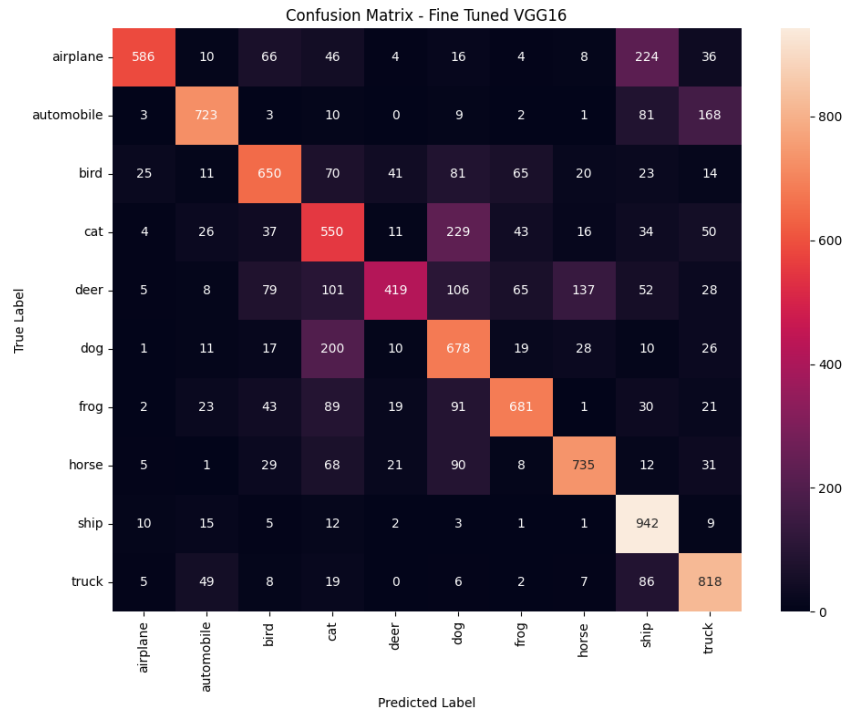


Figure 8: Confusion Matrix – Fine Tuned VGG16

Inference: The model performs strongly on *ship*, *automobile* and *airplane* classes but confuses visually similar animal classes such as *cat*, *dog* and *deer*, which is expected given their similar shapes and textures at 32×32 resolution.

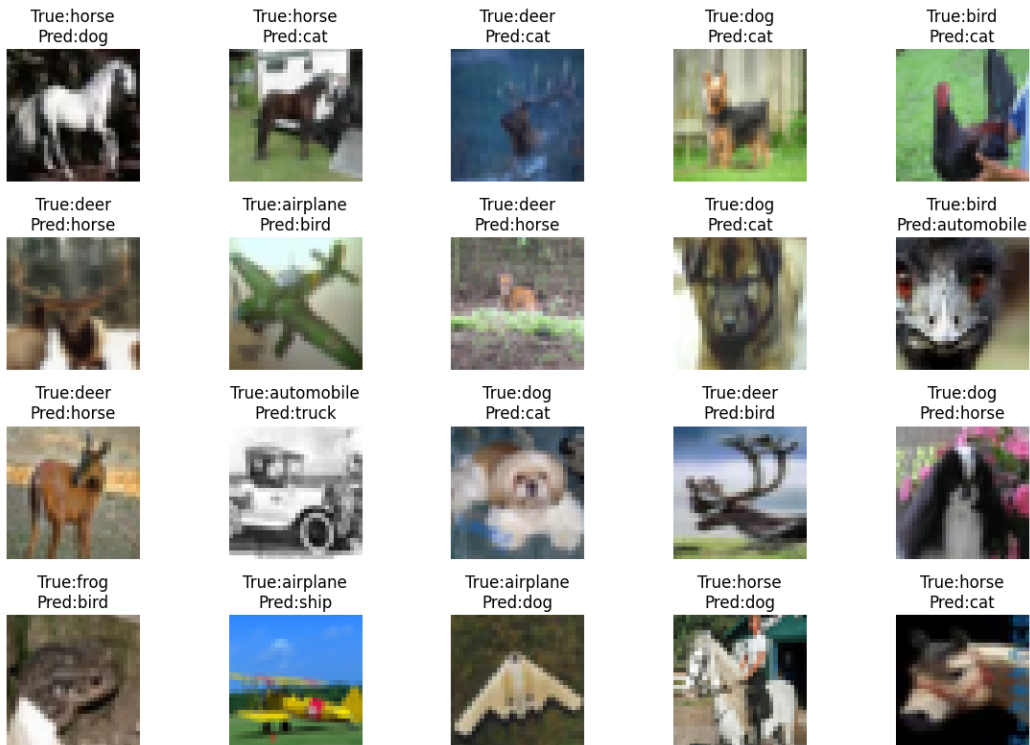


Figure 9: Sample Misclassified Images

Inference: The most common misclassifications were between visually similar categories (like cat predicted as dog, or bird predicted as deer), suggesting that additional data augmentation or higher input resolution could enhance performance.

18. Results

18.1 Performance Metrics (Fine Tuned VGG16)

Metric	Value
Training Accuracy	88.50%
Testing Accuracy	67.82%
Precision	70.58%
Recall	67.82%
F1-score	67.69%
Training Time	157.94 s (64.67 s initial + 93.28 s fine tuning)
Total Parameters	14,848,586

Before fine tuning, the model achieved a best validation accuracy of 60.90%. After unfreezing the `block5` convolution block and fine tuning, the best validation accuracy improved to 71.58%, an improvement of 10.68 percentage points.

18.2 Comparison of CNN Architectures

Model	Parameters	Accuracy (%)	Training Time (s)
LeNet-5	83,126	60.69	49.92
AlexNet	8,590,602	74.53	271.58
VGG16	14,848,586	67.82	157.94
GoogleNet (InceptionV3)	22,329,898	61.78	250.52
ResNet50	24,114,826	18.05	201.74

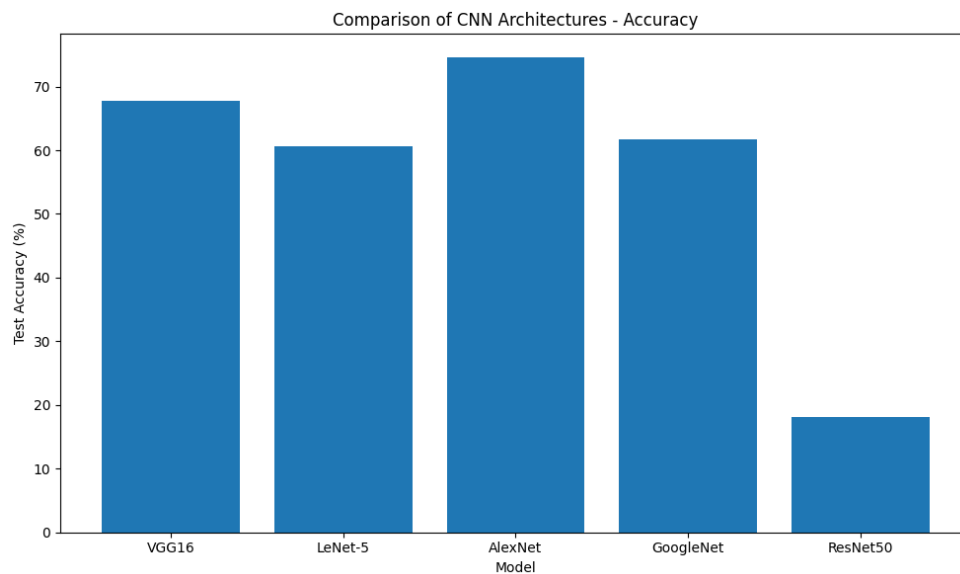


Figure 10: Comparison of CNN Architectures – Test Accuracy

Inference: AlexNet, trained from scratch, achieved the highest test accuracy (74.53%), followed by VGG16 transfer learning (67.82%). ResNet50 performed poorly (18.05%) because 32×32 CIFAR-10

images are much smaller than the 224×224 ImageNet images that it was pretrained on, and only the last convolution block was fine-tuned.

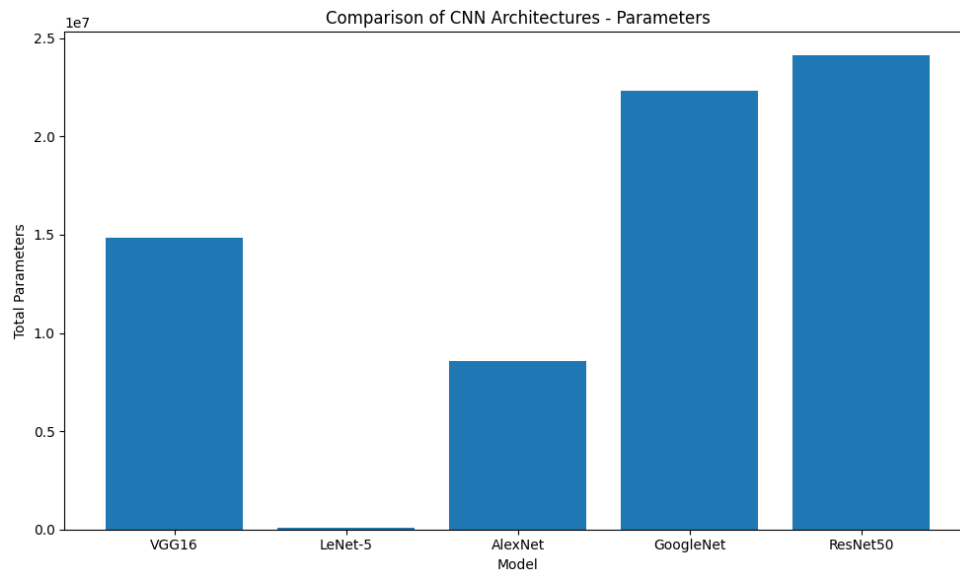


Figure 11: Comparison of CNN Architectures – Total Parameters

Inference: ResNet50 and GoogleNet (InceptionV3 backbone) have the highest number of parameters, and LeNet-5 has the fewest parameters by far. A high parameter count does not necessarily indicate greater accuracy on this low-resolution dataset, as demonstrated by the poor performance of ResNet50, which has the most parameters.

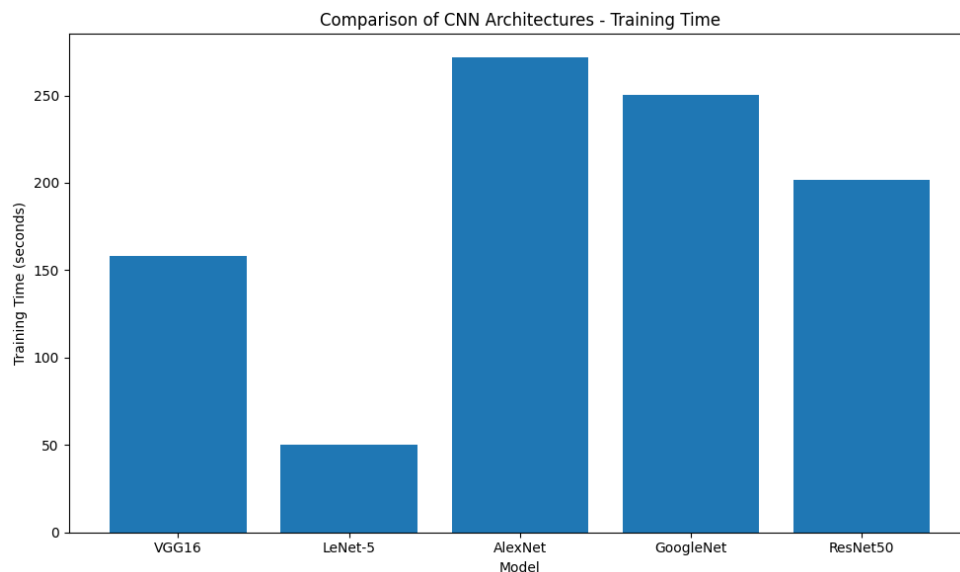


Figure 12: Comparison of CNN Architectures – Training Time

Inference: The pretrained model LeNet-5 appears to be the fastest and most efficient since it had the shallowest architecture, while AlexNet and GoogleNet took the longest time to train. VGG16, in comparison to other pretrained transfer-learning models, demonstrated a decent balance between training time and accuracy.

19. Discussion Questions

1. Why is AlexNet considered a breakthrough in deep learning?

AlexNet won the 2012 ImageNet competition by a huge margin and demonstrated that deep CNNs trained on GPUs can outperform traditional hand-crafted feature based approaches. It introduced the ReLU activation to allow for faster convergence, Dropout to prevent overfitting, GPU-based parallel training to train large models, and data augmentation to improve generalization. All of these improvements sparked a renewed interest in deep learning for computer vision.

2. **Why does VGG16 use only 3×3 convolution filters?**

Using multiple stacked 3×3 filters results in the same effective receptive field as a larger filter (e.g., two 3×3 layers have the same receptive field as one 5×5 layer) while requiring fewer parameters and an additional non-linearity (ReLU) between the layers. This increases the depth and expressiveness of the network while being more amenable to regularization at only a small increase in computational cost.

3. **Explain the advantages of the Inception module.**

The Inception module uses 1×1 , 3×3 and 5×5 convolutions, as well as max pooling, applied in parallel to the same input with their results being concatenated together, thus enabling the network to effectively learn features of different scales. The 1×1 convolutions before the larger kernels in each Inception module can be thought of as dimensionality reduction layers, which drastically reduce the number of parameters and computational complexity, enabling GoogleNet to achieve significantly better accuracy than VGG16 while using way fewer parameters.

4. **What is the purpose of residual learning?**

Residual learning enables a network to learn the residual mapping $F(x) = H(x) - x$ instead of the direct mapping $H(x)$, where the final output is calculated as $F(x) + x$ with the help of a skip connection. This identity shortcut facilitates the gradient flow through the network during backpropagation that avoids the vanishing gradient problem and allows very deep networks (50, 101, or more layers) to be trained effectively and to converge faster.

5. **Differentiate LeNet and ResNet.**

Aspect	LeNet-5	ResNet
Depth	7 layers	50+ layers
Parameters	About 60K	Millions (25.6M for ResNet50)
Key Idea	Basic convolution and pooling layers	Residual (skip) connections
Problem Addressed	Feature learning for simple digit images	Vanishing gradients in very deep networks
Typical Use Case	Handwritten digit recognition (MNIST)	Large-scale image classification (ImageNet)

6. **What is Transfer Learning?**

Transfer Learning is a method where one uses a pretrained model on some huge set of data (say ImageNet) and tweaks it a tiny bit to fit to a smaller dataset. The base convolutional layers are already pretrained and frozen to avoid training them again, since they already can detect many low-level and some mid-level features in images. Thus one does not have to spend time training a CNN from scratch, as it would take significantly longer.

7. **Why is fine tuning required?**

The features, learned by a pretrained model on a source dataset (ImageNet), are general but not perfectly applicable to a new target dataset (CIFAR-10), which may have different classes, resolutions and visual characteristics. Fine tuning unfreezes some of the later convolutional layers and continues their training, along with the new classifier, at a small learning rate. This allows the model to adapt its high-level, task-specific features to the new dataset, typically improving accuracy beyond what is achievable with a frozen base alone.

8. **Explain the difference between dilated convolution and transpose convolution.**

Dilated (atrous) convolution inserts gaps between kernel elements controlled by the dilation rate D to increase the receptive field of a filter without increasing the number of parameters or reducing spatial resolution; it is mainly used for tasks like semantic segmentation where broad

context is needed. Transpose convolution, on the other hand, is a learnable upsampling operation that increases the spatial dimensions of a feature map, effectively reversing the downsampling done by convolution or pooling; it is used in autoencoders, GANs and image generation tasks to reconstruct higher-resolution outputs from compressed representations.

9. Why do pretrained models converge faster?

Pretrained models can begin their optimization process from weights that have already learned useful, general-purpose visual features from a much larger dataset in addition to the target dataset, rather than from randomly initialized weights. As such, they require fewer gradient steps and, consequently, fewer epochs to reach a low loss on the target task than a randomly initialized network.

10. Compare the computational complexity of LeNet and ResNet.

LeNet-5 has about 60K parameters and only 2 convolutional layers, making it very light weight and quick to train, but limited feature complexity. ResNet50 has around 25.6 million parameters with 50 layers of residual connections requiring much more FLOPs, memory and training time per epoch. However, ResNet's depth and skip connections enable learning significantly more rich hierarchical features for much higher accuracy on large-scale complex image classification tasks at the expense of higher computational cost.

20. Additional Exercises

1. Implement transfer learning using VGG16.
2. Repeat the experiment using ResNet50.
3. Compare Adam and SGD optimizers.
4. Compare training with frozen layers and fine tuning.
5. Evaluate the model using Precision, Recall and F1-score.
6. Compare the performance of LeNet, AlexNet and ResNet.

21. References

1. Yann LeCun, Leon Bottou, Yoshua Bengio and Patrick Haffner, *Gradient-Based Learning Applied to Document Recognition*, Proceedings of the IEEE, 1998.
2. Alex Krizhevsky, Ilya Sutskever and Geoffrey Hinton, *ImageNet Classification with Deep Convolutional Neural Networks*, NeurIPS, 2012.
3. Karen Simonyan and Andrew Zisserman, *Very Deep Convolutional Networks for Large-Scale Image Recognition*, ICLR, 2015.
4. Christian Szegedy et al., *Going Deeper with Convolutions*, CVPR, 2015.
5. Kaiming He, Xiangyu Zhang, Shaoqing Ren and Jian Sun, *Deep Residual Learning for Image Recognition*, CVPR, 2016.
6. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
7. TensorFlow Documentation: <https://www.tensorflow.org>
8. Keras Documentation: <https://keras.io>